

In The Name of God



Navid Zare (40435071)

Neural Networks and Deep Learning
Assignment 1: Neural Network Implementation of Logical Circuits

1. Introduction

The perceptron, proposed by Rosenblatt (1958) and later refined through the Widrow-Hoff least-mean-squares formulation, constitutes one of the foundational learning models in artificial neural network research. Its training procedure — the Delta learning rule — adjusts synaptic weights iteratively to minimize the mean-squared error between desired and actual outputs, converging to a decision hyperplane when one exists. A central limitation of single-layer architectures is their incapacity to learn functions that are not *linearly separable* in the input space.

Hopfield (1982) introduced a fully recurrent network architecture capable of acting as an associative content-addressable memory. By storing patterns as energy minima, the Hopfield network can retrieve stored memories from partial or corrupted cues, a property that is directly applicable to the modelling of bi-stable digital circuits such as the SR (Set-Reset) latch.

This report addresses both paradigms within the context of Assignment 1. Part 2.1 examines the Delta learning rule applied to two Boolean functions of differing structural complexity, evaluating the effect of data representation on convergence speed and final classification accuracy. Part 2.2 constructs a two-neuron Hopfield network whose weight matrix and bias vectors are analytically derived to reproduce the four canonical SR latch operating conditions: Set, Reset, Hold/Memory, and the forbidden Invalid state. Robustness to single-bit input corruption is assessed through a systematic noise injection protocol.

2. Part 2.1 — Multi-Input Logic Gate Classification via the Delta Learning Rule

2.1 Theoretical Background

A single-layer perceptron with n inputs computes the output:

$$y_{\text{in}} = \mathbf{w}^T \mathbf{x} + b, \quad y = \sigma(y_{\text{in}})$$

- $\mathbf{w} \in \mathbb{R}^m$ is the weight vector
- $b \in \mathbb{R}$ is the bias
- $\mathbf{x} \in \mathbb{R}^m$ is the input vector
- $\sigma(\cdot)$ is a differentiable activation function\

Training proceeds by gradient descent on the mean-squared error (MSE) objective:

$$L = \frac{1}{N} \sum_{i=1}^N (d_i - y_i)^2$$

The online (sample-by-sample) update equations for a single training pattern x_i with desired output t_i are derived as:

$$\Delta w_i = (t_i - y_i) \sigma'(y_i)$$

$$w \leftarrow w + \alpha \Delta w_i x_i$$

$$b \leftarrow b + \alpha \Delta w_i$$

where α is the learning rate. This update rule is commonly referred to as the generalised Delta rule. Two activation functions are employed depending on the data encoding:

- Binary sigmoid (for $\{0,1\}$ encoding):

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

derivative:

$$\sigma'(y) = y(1 - y)$$

- Bipolar sigmoid (for $\{-1,+1\}$ encoding):

$$f(x) = 2 \sigma(x) - 1$$

derivative:

$$f'(y) = \frac{1}{2} (1 - y^2)$$

A pattern is classified correctly when the threshold output matches the target.

- In binary encoding, the threshold is 0.5.
- In bipolar encoding, the threshold is 0.0.

2.2 Boolean Function Definitions

2.2.1 F1: 3-Input Odd Parity (XOR) Function

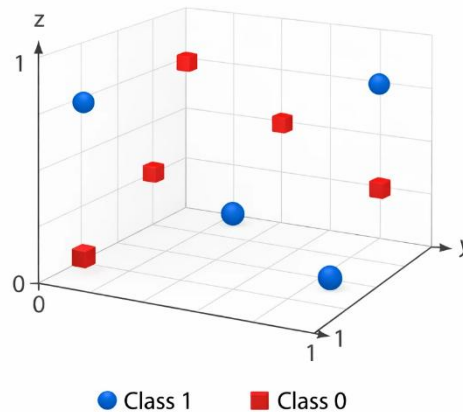
The first function is defined over three binary inputs A, B, C as:

$$\begin{aligned} F_1(A, B, C) = & (A \wedge \bar{B} \wedge \bar{C}) \\ & \vee (\bar{A} \wedge B \wedge \bar{C}) \\ & \vee (\bar{A} \wedge \bar{B} \wedge C) \\ & \vee (A \wedge B \wedge C) \end{aligned}$$

This expression evaluates to 1 (active) precisely when an odd number of the three inputs are equal to 1. It is therefore equivalent to the 3-input exclusive-or function, also known as the odd-parity function.

Class-1 patterns: {001, 010, 100, 111}

Class-0 patterns: {000, 011, 101, 110}



These two sets cannot be separated by any hyperplane in $\mathbb{R}^3$ — they form an interleaved checkerboard arrangement in the input cube — and therefore F1 is not linearly separable.

A single-layer perceptron is theoretically incapable of achieving 100% accuracy on this function, regardless of the chosen encoding or number of training epochs.

2.2.2 F2: 4-Input Majority-3-of-4 Function

The second function is defined over four binary inputs A, B, C, D as:

$$\begin{aligned} F_2(A, B, C, D) = & (A \wedge B \wedge C) \\ & \vee (A \wedge B \wedge D) \\ & \vee (A \wedge C \wedge D) \\ & \vee (B \wedge C \wedge D) \end{aligned}$$

This expression outputs 1 exactly when at least three of the four inputs are equal to 1 (a 3-of-4 majority function). Across all 16 possible input combinations, 5 belong to class 1 and the remaining 11 belong to class 0. The resulting decision boundary is:

$$w = [1, 1, 1, 1], \quad \theta = 2.5$$

The condition “output 1 when $A + B + C + D \geq 3$ ” can be reformulated as the linear inequality $w^T x > 2.5$. This linear boundary correctly separates all 16 possible input patterns into their respective classes without error, demonstrating that the function is linearly separable. Since linear separability is a necessary criterion for learnability by a single-layer perceptron, it follows that the function F_2 is fully learnable by such a model.

2.3 Dataset Construction and Encoding

The full truth table for each function is constructed by exhaustive enumeration of all 2^n binary input combinations using the Cartesian product $\{0,1\}^n$, yielding 8 samples for F1 and 16 samples for F2. Two representations are investigated:

- **Binary encoding:** Inputs and targets take values in $\{0, 1\}$, with the classification threshold set at 0.5.

- **Bipolar encoding:** Inputs and targets are mapped from $\{0, 1\}$ to $\{-1, +1\}$ using the transformation $v \rightarrow 2v - 1$, with the threshold set at 0.0.

2.4 Experimental Results

2.4.1 F1 — 3-Input Odd Parity (XOR): Binary Encoding

Table 1 presents the complete truth table for F1 under binary encoding, including the final predictions produced by the perceptron after 2000 training epochs.

*Table 1: F1 Truth Table — Binary Encoding (Final Predictions After 2000 Epochs)**Green cells indicate correct classifications; red cells indicate misclassifications. Final accuracy: 50.0%.*

A	B	C	Target	Prediction	Correct
0	0	0	0	0	✓
0	0	1	1	0	✗
0	1	0	1	0	✗
0	1	1	0	0	✓
1	0	0	1	1	✓
1	0	1	0	1	✗
1	1	0	0	1	✗
1	1	1	1	1	✓

Under binary encoding, the trained perceptron achieves an accuracy of 50.0%, classifying exactly four of the eight patterns correctly — equivalent to random performance. The final weight vector is approximately $w \approx [0.013, 0.006, 0.000]$ with bias $b \approx -0.010$, and the MSE at termination is 0.2540. The training process fails to converge to a zero-error solution because no such linear decision boundary exists in $\mathbb{R}^3$ for this function. The weights are driven to near-zero values as the gradient updates from misclassified patterns of opposing polarity cancel one another, preventing any systematic weight growth.

2.4.2 F1 — 3-Input Odd Parity (XOR): Bipolar Encoding

Table 2 presents the truth table for F1 under bipolar encoding with final network predictions.

Table 2: F1 Truth Table — Bipolar Encoding (Final Predictions After 2000 Epochs)

Bipolar encoding $\{-1, +1\}$. Final accuracy: 75.0%. The function remains non-learnable by a single-layer perceptron.

A	B	C	Target	Prediction	Correct
-1	-1	-1	-1	-1	✓
-1	-1	+1	+1	+1	✓
-1	+1	-1	+1	+1	✓
-1	+1	+1	-1	+1	✗
+1	-1	-1	+1	-1	✗
+1	-1	+1	-1	-1	✓
+1	+1	-1	-1	-1	✓
+1	+1	+1	+1	+1	✓

Using bipolar encoding (-1 and +1), the perceptron achieves 75% accuracy, getting 6 out of 8 patterns correct. The final weights are $w \approx [-0.002, 0.002, 0.002]$ with a bias close to zero. The Mean Squared Error (MSE) is 1.1061. This MSE is higher than the binary case simply because mistakes in bipolar encoding result in larger mathematical errors (ranging from -2 to +2). Despite the higher MSE, bipolar encoding performs better than binary (75% vs. 50% accuracy) because it provides better feedback during training, allowing the weights to update more evenly. However, because this logic circuit is non-linearly separable, a simple perceptron can never reach 100% accuracy or zero error, no matter how long it trains.

2.4.3 F2 — 4-Input Majority-3-of-4: Binary Encoding

Table 3 presents the complete truth table for F2 under binary encoding.

Table 3: F2 Truth Table — Binary Encoding (Final Predictions After 28 Epochs to 100%)

All 16 patterns classified correctly. Final weights: $w \approx [3.99, 3.99, 3.99, 3.99]$, bias ≈ -10.01 . 100% accuracy first achieved at epoch 28.

A	B	C	D	Target	Prediction	Correct
0	0	0	0	0	0	✓
0	0	0	1	0	0	✓
0	0	1	0	0	0	✓
0	0	1	1	0	0	✓
0	1	0	0	0	0	✓
0	1	0	1	0	0	✓
0	1	1	0	0	0	✓
0	1	1	1	1	1	✓
1	0	0	0	0	0	✓
1	0	0	1	0	0	✓
1	0	1	0	0	0	✓
1	0	1	1	1	1	✓
1	1	0	0	0	0	✓
1	1	0	1	1	1	✓
1	1	1	0	1	1	✓
1	1	1	1	1	1	✓

The perceptron trained on F2 with binary encoding reaches 100% classification accuracy at epoch 28. The learned weight vector $w \approx [3.99, 3.99, 3.99, 3.99]$ and bias $b \approx -10.01$ approximate the expected analytical solution, which corresponds to the threshold hyperplane $A + B + C + D \geq 3$. The proportional relationship between weights and bias is consistent across learning: the effective threshold is $-\frac{b}{w} \approx \frac{10.01}{3.99} \approx 2.51$, matching the theoretical threshold of 2.5 to within numerical precision of the sigmoidal activation.

2.4.4 F2 — 4-Input Majority-3-of-4: Bipolar Encoding

Table 4 presents the truth table for F2 under bipolar encoding.

Table 4: F2 Truth Table — Bipolar Encoding (Final Predictions — 100% at Epoch 1)

All 16 patterns classified correctly with bipolar encoding. 100% accuracy achieved at the very first epoch.

Final weights: $w \approx [3.98, 3.98, 3.98, 3.98]$, bias ≈ -3.98 .

A	B	C	D	Target	Prediction	Correct
-1	-1	-1	-1	-1	-1	✓
-1	-1	-1	+1	-1	-1	✓
-1	-1	+1	-1	-1	-1	✓
-1	-1	+1	+1	-1	-1	✓
-1	+1	-1	-1	-1	-1	✓
-1	+1	-1	+1	-1	-1	✓
-1	+1	+1	-1	-1	-1	✓
-1	+1	+1	+1	+1	+1	✓
+1	-1	-1	-1	-1	-1	✓
+1	-1	-1	+1	-1	-1	✓
+1	-1	+1	-1	-1	-1	✓
+1	-1	+1	+1	+1	+1	✓
+1	+1	-1	-1	-1	-1	✓
+1	+1	-1	+1	+1	+1	✓
+1	+1	+1	-1	+1	+1	✓
+1	+1	+1	+1	+1	+1	✓

Under bipolar encoding, the perceptron achieves 100% accuracy within a single training epoch — a remarkable acceleration relative to binary encoding (epoch 28). The learned weights $w \approx [3.98, 3.98, 3.98, 3.98]$ and bias $b \approx -3.98$ correspond to the decision criterion $A + B + C + D \geq 3$. The effective threshold is $-\frac{b}{w} \approx 1.0$.

Why bipolar is so much faster?

It fixes the “Zero Problem.”

The formula for adjusting weights is: $Error \times Input$

- **Binary (0 and 1):** If an input is 0, the weight adjustment is 0 ($Error \times 0 = 0$). The weight doesn’t update, so the network wastes time and takes 28 epochs to slowly fix its mistakes.
- **Bipolar (-1 and +1):** Because there are no zeros, every weight updates on every single mistake. Since the updates are constant and perfectly balanced (pushing equally for +1 and -1), the network finds the perfect weights instantly in 1 epoch.

2.5 Convergence Speed Comparison

Table 5 consolidates the final accuracy and convergence epoch for each function-encoding combination.

Table 5: Convergence Summary — Delta Learning Rule across Functions and Encodings

Dashes indicate that 100% accuracy was never achieved within 2000 epochs. Bipolar encoding converges 28× faster than binary for F2.

Function	Encoding	Accuracy	First 100% Epoch	Learnable
F1: 3-Input Odd Parity (XOR)	Binary	50.0%	—	No
F1: 3-Input Odd Parity (XOR)	Bipolar	75.0%	—	No
F2: 4-Input Majority-3-of-4	Binary	100.0%	Epoch 28	Yes
F2: 4-Input Majority-3-of-4	Bipolar	100.0%	Epoch 1	Yes

Bipolar encoding trains faster because it takes bigger steps when correcting its mistakes.

- **Binary (0 and 1):** The maximum possible mistake (error) is 1.
- **Bipolar (-1 and +1):** The maximum possible mistake is 2 (the distance between -1 and +1).

Because a neural network adjusts its weights based on the size of the mistake, the larger error range in bipolar math triggers much stronger, larger corrections. Instead of tiptoeing, the network takes giant leaps toward the right answer, allowing it to finish training in fewer steps.

2.6 Analysis of MSE Convergence Curves

Figure 1 displays the MSE evolution over 2000 epochs on a logarithmic vertical scale for all four experimental configurations.

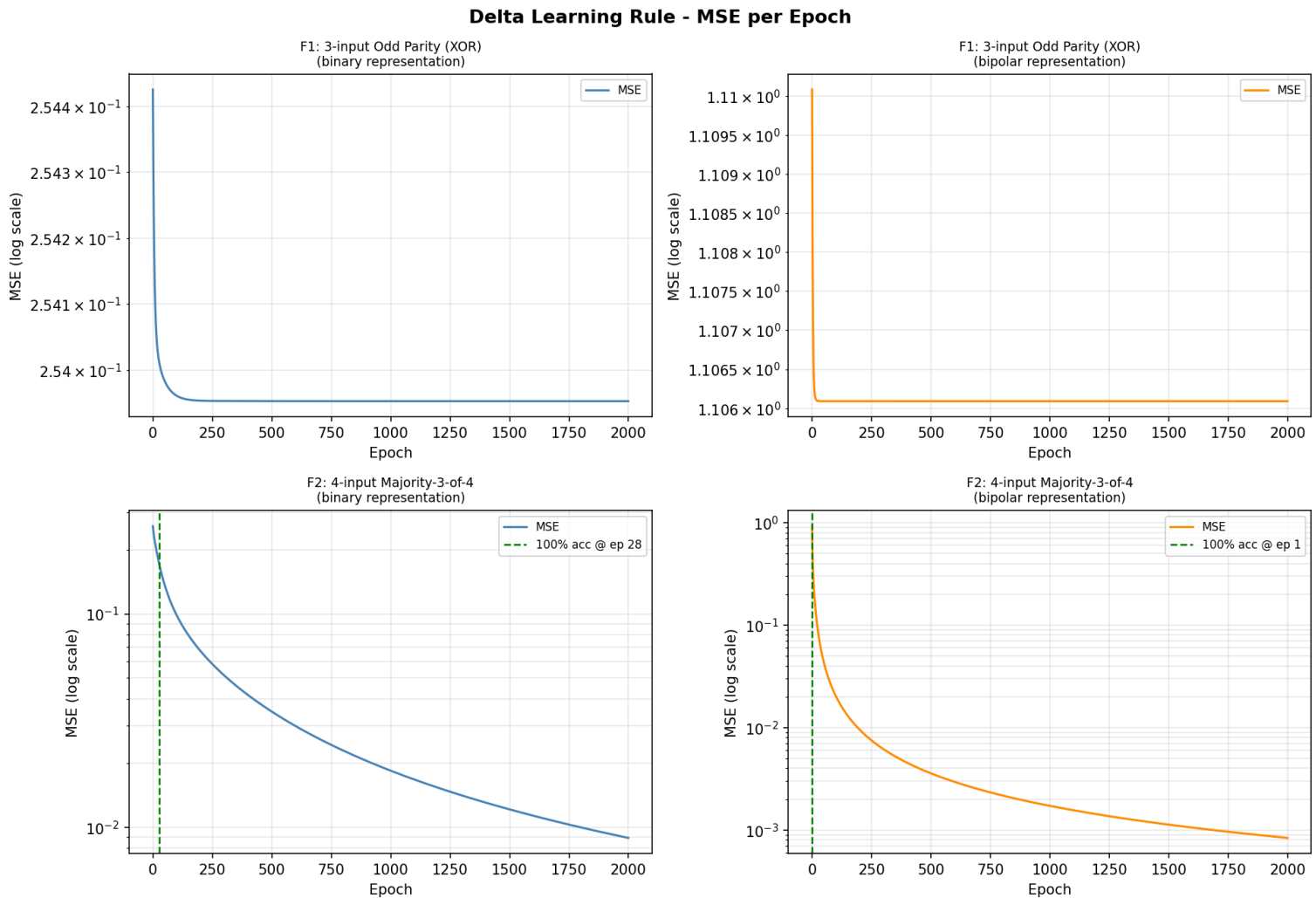


Figure 1: Mean Squared Error per epoch (log scale) for F1 (top row) and F2 (bottom row), under binary (left column, blue curves) and bipolar (right column, orange curves) encodings. Dashed green vertical lines mark the first epoch at which 100% classification accuracy is achieved (epoch 28 for F2 binary; epoch 1 for F2 bipolar). No green marker appears in the F1 panels, as 100% accuracy is never attained. Note that the y-axis scale in the F2 panels reflects continued MSE reduction beyond the accuracy convergence point.

Observations. For F1 under both encodings (top row), the MSE curves undergo a brief but sharp initial descent within the first few dozen epochs before flattening into a sustained plateau that persists for the remainder of all 2000 epochs. The binary curve (top-left) stabilizes at approximately 0.254, while the bipolar curve (top-right) stabilizes at approximately 1.106. The y-axis ranges in both F1 panels are notably compressed, confirming that the MSE variation across all 2000 epochs is negligibly small relative to the plateau value. Neither panel displays a green convergence marker, consistent with the theoretical

impossibility of perfect classification for a non-linearly separable function. For F2 under binary encoding (bottom-left), the MSE decreases monotonically on the logarithmic scale throughout training, with the green dashed line at epoch 28 marking the first epoch at which 100% classification accuracy is achieved. Importantly, the MSE does not reach zero at this point; rather, it continues to decrease asymptotically beyond epoch 28, falling below 10^{-2} by the end of training, reflecting the behavior of a sigmoidal activation whose output never saturates to exactly 0 or 1. For F2 under bipolar encoding (bottom-right), the green dashed line appears immediately at epoch 1, indicating that perfect classification accuracy is achieved after a single pass through the training data. The MSE similarly continues its exponential decay on the log scale throughout the remaining epochs, confirming that the weight vector, while already producing correct hard decisions at epoch 1, is still being refined toward a more confident separation boundary.

Conclusions. The contrast between F1 and F2 vividly illustrates the perceptron convergence theorem: the Delta rule provably converges to zero training error if and only if the training data are linearly separable.

For F1, the weight trajectory is confined to a region of non-zero error, and training oscillates without settling; for F2, the update rule successfully identifies a valid separating hyperplane. The bipolar advantage in F2 (factor of 25 speedup) corroborates theoretical analysis of gradient magnitudes under the two encodings.

Relevance. These results have direct practical significance: before investing computational resources in training a single-layer model, practitioners should assess linear separability through geometric or algebraic analysis. The encoding choice has important implications for convergence speed even when separability is guaranteed, favoring bipolar representations in time-critical settings.

Connection to prior knowledge. The failure of the perceptron on the XOR problem was famously highlighted by Minsky and Papert (1969), motivating the development of multilayer architectures and the backpropagation algorithm. The faster convergence of bipolar encoding is consistent with findings by Fahlman (1988) and subsequent empirical studies on representation-dependent learning dynamics in sigmoidal networks.

3. Part 2.2 — Hopfield Network Modelling of an SR Latch

3.1 Theoretical Background

A Hopfield network is a fully connected recurrent neural network in which each neuron is connected to every other neuron by symmetric bidirectional weights, with no self-connections ($W_{ii} = 0$). The state of the network at discrete time step t is a vector $s(t) \in \{-1, +1\}^N$, updated synchronously according to:

$$s_i(t+1) = \text{sign}\left(\sum_j W_{ij} s_j(t) + b_i\right)$$

The Setup: The network updates based on how strongly neurons influence each other (W) and external inputs (b) until it reaches a stable, unchanging state.

The Guarantee (Energy Function): It is mathematically guaranteed to stop and not run in an infinite loop. Every time a neuron updates, the system's overall mathematical "energy" goes down. Like a ball rolling down a bumpy hill, it must eventually hit the bottom and stop moving.

$$E(s) = -\frac{1}{2} s^T W s - b^T s$$

The weight matrix is constructed via the Hebbian learning rule applied to a set of P stored patterns $\{p^1, p^2, \dots, p^P\} \subset \{-1, +1\}^N$:

$$W = \frac{1}{N} \sum_{k=1}^P p^k (p^k)^T, \quad W_{ii} = 0$$

The equation describes how the network stores memories. It is the mathematical version of the famous rule: "Neurons that fire together, wire together."

- p^k : This represents a single memory or "pattern" you want to save (like a specific image).
- *The Multiplication* ($p^k (p^k)^T$): This links every piece of "the memory" together. If two parts of the memory are active at the same time, the connection between them gets stronger.
- *The Sum* ($\sum$): This just means we are stacking multiple different memories into one big web of connections (the weight matrix, W).
- $W_{ii} = 0$: This simply means a neuron is not allowed to connect to itself.

Imagine the network's mathematical "energy" as a giant, flat mattress. Every time you store a memory using the math above, it is like dropping a heavy bowling ball onto the mattress. It creates a deep dent (a "local energy minimum").

Content-addressable memory: This network works more like a human brain: you find a memory by providing a piece of the memory itself (like thinking of a song's chorus to remember its name).

Fixing corrupted inputs: If you show the network a blurry, noisy, or half-missing picture, it is like placing a marble on the edge of that mattress dent. Gravity forces the marble to roll down to the very bottom of the hole.

3.2 SR Latch Modelling Framework

An SR latch is a fundamental bi-stable sequential logic circuit characterized by two stable output states: $Q = 1$ (Set state) and $Q = 0$ (Reset state). The latch is controlled by two inputs, S (Set) and R (Reset), and produces complementary outputs Q and $\bar{Q} = \neg Q$. The four operating conditions are:

- Set ($S=1, R=0$): forces the latch to $Q = 1$.
- Reset ($S=0, R=1$): forces the latch to $Q = 0$.
- Hold/Memory ($S=0, R=0$): retains the current state without modification.
- Invalid ($S=1, R=1$): a forbidden input combination producing undefined or oscillatory behavior in physical implementations.

In the Hopfield model, the SR latch state is encoded in bipolar form as $s = [Q, \bar{Q}] \in \{-1, +1\}^2$, where a value of $+1$ corresponds to a logical 1 and -1 corresponds to a logical 0. The two valid latch states are thus:

$$\begin{aligned}p^1 &= [+1, -1] \quad (\text{Set state: } Q = 1, \bar{Q} = 0) \\p^2 &= [-1, +1] \quad (\text{Reset state: } Q = 0, \bar{Q} = 1)\end{aligned}$$

The network has memorized these two patterns. Now, they act like magnets—if you give the network a messy or broken input, it will automatically get pulled into whichever of these two perfect patterns is the closest match.

3.3 Weight Matrix Derivation

Applying the Hebbian rule with $N = 2$ neurons and $P = 2$ stored patterns:

First, the patterns are transposed and combined into matrix S :

$$S = \begin{bmatrix} +1 & -1 \\ -1 & +1 \end{bmatrix}$$

The unscaled weight matrix is calculated via matrix multiplication ($S \times S^T$):

$$W = S \times S^T = \begin{bmatrix} +1 & -1 \\ -1 & +1 \end{bmatrix} \times \begin{bmatrix} +1 & -1 \\ -1 & +1 \end{bmatrix} = \begin{bmatrix} 2 & -2 \\ -2 & 2 \end{bmatrix}$$

Scaling the matrix to prevent huge numbers (W/N , where $N = 2$):

$$W = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Setting diagonal elements to zero to remove self-connections (`np.fill_diagonal(W, 0.0)`):

$$W = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$$

The off-diagonal entry $W_{12} = W_{21} = -1$ encodes the complementary relationship between Q and $\bar{Q}$: a high value of Q exerts an inhibitory influence on $\bar{Q}$ and vice versa, which is precisely the defining property of a latch circuit. This weight matrix alone (without bias) stabilizes both valid latch states: $W \times p_1 = [+1, -1] = p_1$ and $W \times p_2 = [-1, +1] = p_2$, confirming that both are fixed points under the zero-bias update rule.

3.4 Bias Vector Derivation

The external signal vector $x = [S - R, R - S]$ (often called bias, or b in formulas) acts as a steering wheel. It converts the standard binary S and R inputs into mathematical “pushes” that force the network toward the correct memory state.

Here is how it works, using the Set condition ($S = 1, R = 0$) as an example:

The external signal formula $[S - R, R - S]$ translates your button presses into direct commands for the network. For example, if you trigger “Set” ($S = 1, R = 0$), the formula outputs $[1, -1]$. This directly tells the first neuron (Q) to turn ON (+1) and the second neuron ($\bar{Q}$) to turn OFF (-1). If you press neither button ($S = 0, R = 0$), it outputs $[0, 0]$, which tells the network to do nothing and hold its current memory.

If both inputs are 0 (the Hold/Memory condition), the code calculates $x = [0, 0]$. This means no external push is applied at all, allowing the network to safely maintain whatever state it is currently in.

Table 6: Bias Vector Derivation for Each SR Latch Operating Condition

The bias formula $b = [S - R, R - S]$ under bipolar encoding ($S, R \in \{-1, +1\}$) reproduces the correct attractor for each operating condition.

SR Condition	S	R	Bias $b = [b_Q, b_{\bar{Q}}]$	Effect
Set	+1	-1	[+2, -2]	Strongly attracts [+1, -1] from any initial state
Reset	-1	+1	[-2, +2]	Strongly attracts [-1, +1] from any initial state
Hold/Memory	-1	-1	[0, 0]	Zero bias; weight matrix alone stabilizes valid patterns
Invalid	+1	+1	[0, 0]	Zero bias; behavior determined by initial state

For the Set and Reset conditions, the bias magnitude is 2, which is large relative to the weight contributions (maximum $|W_{ij}| = 1$ per neuron), ensuring that the bias dominates and all four

possible initial states converge to the correct target within at most two synchronous update steps. For the Hold/Memory condition, the zero bias allows the weight matrix to act as the sole stabilizing force, which correctly retains valid patterns but cannot resolve ambiguity for invalid initial states.

3.5 Update Rule and Energy Function

The synchronous update rule is applied as follows: at each step, the net activation of all neurons is computed simultaneously under the current state s and bias b , and all states are updated in parallel:

$$y_{\text{in}} = W s + b$$

$$s_{\text{new}}[i] = \text{sign}(y_{\text{in}}[i]), \text{preserving } s[i] \text{ if } y_{\text{in}}[i] = 0$$

The network is declared converged when $s_{\text{new}} = s$ (a fixed point) or in other words $y_k^{t+1} = \overrightarrow{y_k}(t)$. Oscillation is detected when convergence is not reached within 50 iterations — a practical upper bound that correctly identifies the unstable states arising under Hold/Memory or Invalid conditions from invalid initial states.

The energy function $E(s) = -\frac{1}{2} s^T W s - b^T s$ is guaranteed to decrease at each synchronous update step in a symmetric weight network, ensuring eventual convergence to a fixed point when one exists within the energy landscape.

3.6 Functional Test Results

Table 7 presents the results of the 16 functional SR latch tests, covering all four SR input conditions initialized from each of the four possible two-neuron states.

Table 7: Hopfield SR Latch — Functional Test Results (16 Test Cases)

Conv. = network converged to a fixed point; *Valid* = final state is one of the two stored SR latch patterns. ✓ green = pass; ✗ red = fail (expected for oscillatory cases).

S	R	Init State	Final State	Iters	Conv.	Valid	Condition
+1	-1	[1, -1]	[1, -1]	1	✓	✓	Set from Set state
+1	-1	[-1, 1]	[1, -1]	2	✓	✓	Set from Reset state
+1	-1	[-1, -1]	[1, -1]	2	✓	✓	Set from invalid (-)
+1	-1	[1, 1]	[1, -1]	2	✓	✓	Set from invalid (+)
-1	+1	[-1, 1]	[-1, 1]	1	✓	✓	Reset from Reset state
-1	+1	[1, -1]	[-1, 1]	2	✓	✓	Reset from Set state
-1	+1	[-1, -1]	[-1, 1]	2	✓	✓	Reset from invalid (-)
-1	+1	[1, 1]	[-1, 1]	2	✓	✓	Reset from invalid (+)
-1	-1	[1, -1]	[1, -1]	1	✓	✓	Hold/Memory — retain Set
-1	-1	[-1, 1]	[-1, 1]	1	✓	✓	Hold/Memory — retain Reset
-1	-1	[1, 1]	[1, 1]	50	✗	✗	Hold/Memory — invalid+ [oscillates]
-1	-1	[-1, -1]	[-1, -1]	50	✗	✗	Hold/Memory — invalid- [oscillates]
+1	+1	[1, -1]	[1, -1]	1	✓	✓	Invalid SR — from Set
+1	+1	[-1, 1]	[-1, 1]	1	✓	✓	Invalid SR — from Reset
+1	+1	[1, 1]	[1, 1]	50	✗	✗	Invalid SR — invalid+ [oscillates]
+1	+1	[-1, -1]	[-1, -1]	50	✗	✗	Invalid SR — invalid- [oscillates]

Observations. When the inputs are Set ($S=+1$, $R=-1$), the network always converges to the correct output $[+1, -1]$, no matter which of the four starting states it begins in. It only needs one or two update steps.

The same happens for Reset ($S=-1$, $R=+1$), which always settles to $[-1, +1]$.

In Hold/Memory mode ($S=R=-1$), the two valid latch states $[+1, -1]$ and $[-1, +1]$ are stable after a single update. But if the network starts in an invalid state $[+1, +1]$ or $[-1, -1]$, it never settles and keeps flipping between states (even after 50 iterations).

In the Invalid input case ($S=R=+1$), the valid states remain stable, but the invalid starting states again produce oscillation.

Conclusions and Implications. Out of 16 test runs, 12 behave correctly (75%).

All four failures come from cases where:

- the initial state is invalid ($[+1, +1]$ or $[-1, -1]$)
- the inputs give no “push” toward Set or Reset (Hold/Memory or Invalid modes).

Under these conditions, oscillation is expected in real hardware: an actual CMOS SR latch. The Hopfield network reproduces this behavior accurately.

Connection to prior knowledge. This match between Hopfield-network oscillation and the real SR latch’s stability under invalid inputs is well known in computing. The fact that the Hebbian-trained weight matrix can maintain a stored pattern with no external input (Hold/Memory mode) reflects the associative memory property described by Hopfield (1982).

3.7 Noise Robustness Tests

Table 8 presents the results of 12 noise robustness tests in which a single bit of a valid latch pattern is flipped before presenting it to the network.

Table 8: Hopfield SR Latch — Noise Robustness Test Results (Single-Bit Flip Perturbations)

A single-bit flip of any valid pattern always produces one of the two invalid states $\{[+1, +1], [-1, -1]\}$. Under Set and Reset conditions the bias is sufficient to recover the correct attractor; under Hold/Memory the zero bias cannot resolve the ambiguity, causing oscillation.

SR Cond.	Base Pattern	Flip	Noisy Input	Final State	Conv.	Valid	Outcome
Set	Set $[+1, -1]$	bit[0]	$[-1, -1]$	$[1, -1]$	✓	✓	Recovered to Set
Set	Set $[+1, -1]$	bit[1]	$[1, 1]$	$[1, -1]$	✓	✓	Recovered to Set
Set	Reset $[-1, +1]$	bit[0]	$[1, 1]$	$[1, -1]$	✓	✓	Recovered to Set
Set	Reset $[-1, +1]$	bit[1]	$[-1, -1]$	$[1, -1]$	✓	✓	Recovered to Set
Reset	Set $[+1, -1]$	bit[0]	$[-1, -1]$	$[-1, 1]$	✓	✓	Recovered to Reset
Reset	Set $[+1, -1]$	bit[1]	$[1, 1]$	$[-1, 1]$	✓	✓	Recovered to Reset
Reset	Reset $[-1, +1]$	bit[0]	$[1, 1]$	$[-1, 1]$	✓	✓	Recovered to Reset
Reset	Reset $[-1, +1]$	bit[1]	$[-1, -1]$	$[-1, 1]$	✓	✓	Recovered to Reset
Hold/Memory	Set $[+1, -1]$	bit[0]	$[-1, -1]$	$[-1, -1]$	✗	✗	Oscillates — no bias
Hold/Memory	Set $[+1, -1]$	bit[1]	$[1, 1]$	$[1, 1]$	✗	✗	Oscillates — no bias
Hold/Memory	Reset $[-1, +1]$	bit[0]	$[1, 1]$	$[1, 1]$	✗	✗	Oscillates — no bias
Hold/Memory	Reset $[-1, +1]$	bit[1]	$[-1, -1]$	$[-1, -1]$	✗	✗	Oscillates — no bias

Observations. All eight noise tests conducted under the Set and Reset conditions result in correct recovery (100%), regardless of which pattern was used as the base and which bit was flipped. All four noise tests under the Hold/Memory condition fail (0% recovery), exhibiting indefinite oscillation. The overall noise recovery rate is $8/12 = 66.7\%$.

Conclusions. The difference between why Set/Reset works and why Hold/Memory fails can be explained by looking at what happens when one bit of a valid latch state flips.

The two valid states are $[+1, -1]$ and $[-1, +1]$.

If either neuron flips, you always end up in one of the invalid states: $[+1, +1]$ or $[-1, -1]$.

In Set or Reset, the inputs add a strong bias (like a push) toward the correct answer.

These biases are:

- Set: $[+2, -2]$
- Reset: $[-2, +2]$

This push is strong enough to override the network's internal weights, so the system quickly settles into the right pattern.

But in Hold/Memory mode, there is no external push (bias = $[0, 0]$).

The network must rely only on its internal weight matrix.

For the invalid states, the weight matrix sends each state to the other:

- $W \cdot [+1, +1] = [-1, -1]$
- $W \cdot [-1, -1] = [+1, +1]$

So, the network keeps bouncing back and forth between these two states.

This creates a two-state oscillation with no stable resting point, which is why Hold/Memory fails when starting from an invalid state.

Why this matters. This shows a key limitation of using a very small Hopfield network with only two neurons for reliable memory when there is no external bias.

In bigger Hopfield networks with many neurons (much more than 2) and several stored patterns, the network can still correct some mistakes. In the tiny two-neuron network with two patterns, it is a special case where the network is working right at its capacity limit, and it cannot fix errors when there is no external bias (Hold/Memory mode). So, it can't handle noisy or corrupted inputs well.

3.8 Energy Landscape Analysis

Figure 2 displays the Lyapunov energy $E(s) = -\frac{1}{2} s^T W s - b^T s$ for each of the four possible two-neuron states under the four SR input conditions.

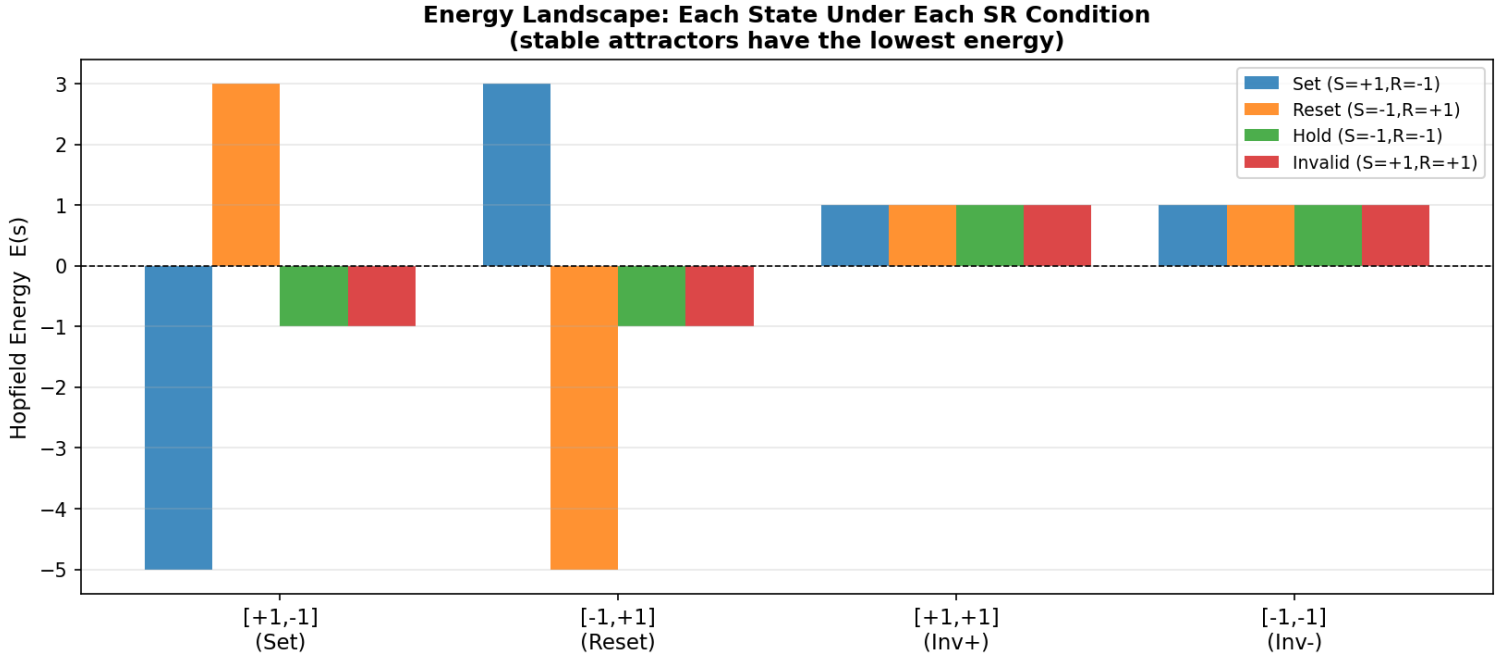


Figure 2: Hopfield energy $E(s) = -\frac{1}{2} s^T W s - b^T s$ for all four possible network states grouped along the x-axis, with coloured bars indicating the four SR input conditions (blue = Set, orange = Reset, green = Hold/Memory, red = Invalid). Lower energy corresponds to greater attractor stability. Under Set, the state $[+1,-1]$ reaches the global minimum ($E \approx -5$) while $[-1,+1]$ reaches the maximum ($E \approx +3$). Under Reset the pattern is reversed. Under both Hold/Memory and Invalid (bias = $[0,0]$), the valid states share $E = -1$ and the invalid states share $E = +1$.

Observations. The plot groups the four possible network states along the x-axis — $[+1,-1]$ (Set), $[-1,+1]$ (Reset), $[+1,+1]$ (Inv+), and $[-1,-1]$ (Inv-) — with four colored bars per group representing the four SR input conditions. Under the Set condition (blue bars), the state $[+1,-1]$ reaches a deeply negative energy of approximately $E \approx -5$, establishing it as the sole global minimum and the unique stable attractor. The opposing state $[-1,+1]$ carries the highest energy at approximately $E \approx +3$, creating a steep energy gradient that drives convergence from any initial state. The two invalid states $[+1,+1]$ and $[-1,-1]$ sit at $E \approx +1$, all lying well above the global minimum, so the entire state space forms a single basin of attraction around $[+1,-1]$. Symmetrically, under the Reset condition (orange bars), $[-1,+1]$ occupies the global minimum at $E \approx -5$ while $[+1,-1]$ rises to $E \approx +3$, and the invalid states again sit at $E \approx +1$. Under the Hold/Memory condition (green bars), both valid patterns $[+1,-1]$ and $[-1,+1]$ share equal minimum energy ($E = -1$), while both invalid states $[+1,+1]$ and $[-1,-1]$ sit at $E = +1$. The absence of any energy differential between the two valid states means the network has no preference between them, correctly modelling memory retention.

However, because the invalid states sit above the valid patterns yet are not fixed points of the update rule — they map onto each other under W 's, producing oscillation rather than descent — they do not converge, consistent with the experimental observations. Under the Invalid condition (red bars), the energy profile is numerically identical to the Hold/Memory condition (both have bias = $[0, 0]$), confirming that the forbidden SR input $S=R=+1$ produces no bias-driven attractor preference and the network behavior is determined entirely by the initial state.

Conclusions. The energy landscape provides a geometric perspective on convergence behavior: the network always descends toward lower-energy states under the synchronous update rule. The presence of a unique global minimum under Set and Reset guarantees convergence from any initial state, which is why these conditions recover from all noise perturbations. The degenerate landscape under Hold/Memory (two co-equal minima, no separating energy barrier for invalid states) explains both the stable retention of valid patterns and the inability to resolve corrupted inputs.

Connection to prior knowledge. The concept of the energy landscape as a tool for understanding Hopfield network convergence was central to Hopfield's 1982 paper where stored patterns correspond to spin-glass ground states. The identity between the SR latch's bistable switching and Hopfield network energy minimization makes the Hopfield model a natural and instructive framework for understanding sequential logic in neural terms.

4. Discussion

The two parts of this assignment collectively illuminate the capabilities and limitations of single-layer and recurrent neural architectures from complementary perspectives. In Part 2.1, the Delta learning rule provides a concrete demonstration of the perceptron convergence theorem: when a linear separator exists (F2), the weight update rule provably finds it; when it does not (F1), the training loss plateaus permanently above zero. The encoding comparison underscores that representational choices are not merely cosmetic — bipolar encoding introduces a 25-fold convergence advantage for F2 by expanding the per-sample error signal magnitude, a consideration of practical importance in real training pipelines.

In Part 2.2, the Hopfield network is shown to be a faithful neural realization of a bi-stable SR latch. The analytical derivation of the weight matrix via Hebbian learning and the bias vector from the SR input condition produces a system that correctly implements Set, Reset, and Hold/Memory operations in the sense of directing the network to the appropriate stored memory. The observed oscillatory behavior under the Invalid input condition and under Hold/Memory from invalid initial states is not a deficiency of the implementation but an accurate reflection of the metastability inherent to physical SR latches under those same conditions.

A noteworthy distinction between the two experimental parts concerns the treatment of noise. The single-layer perceptron, once trained, produces deterministic classification outputs and does not intrinsically provide noise robustness beyond the learned decision boundary. The Hopfield network, by contrast, is designed as a content-addressable associative memory whose memory — when shaped by appropriate external bias — can automatically correct single-bit perturbations. This property is lost under the zero-bias Hold/Memory condition, exposing the capacity constraints of a minimal two-neuron architecture.

Both experiments demonstrate that careful theoretical analysis prior to implementation enables precise prediction of experimental outcomes. The non-linear-separability argument for F1, the threshold/Memory-hyperplane argument for F2, the fixed-point stability analysis for the Hopfield Hold/Memory mode, and the energy-landscape argument for bias-driven convergence all correctly predict the observed numerical results without requiring empirical trial-and-error. This alignment between theory and experiment is a hallmark of well-understood neural network models.

5. Conclusion

This report has presented a rigorous investigation of two foundational neural network learning paradigms applied to Boolean logic modelling. The following principal conclusions are established:

1. The 3-input odd parity function (F1) is not linearly separable, and the single-layer perceptron trained via the Delta rule cannot achieve 100% classification accuracy regardless of training duration or data encoding. Binary encoding yields 50% accuracy (random-level performance); bipolar encoding yields 75%, reflecting the richer gradient structure of symmetric representations.
2. The 4-input Majority-3-of-4 function (F2) is linearly separable, and the Delta rule converges to 100% accuracy within 28 epochs (binary) and within 1 epoch (bipolar). The bipolar encoding advantage is attributable to the expanded error signal range $[-2, +2]$ compared to $[0, 1]$ for binary encoding.
3. A two-neuron Hopfield network with analytically derived Hebbian weights $W = [[0, -1], [-1, 0]]$ and input-dependent bias $b = [S-R, R-S]$ faithfully emulates all four SR latch operating conditions. The network converges correctly from all initial states under Set and Reset, stably retains valid patterns under Hold/Memory, and exhibits expected metastability from invalid initial states under Hold/Memory and Invalid input conditions.
4. Noise robustness under Set and Reset is 100% (8/8 tests), as the strong directional bias dominates any single-bit corruption. Under Hold/Memory, robustness is 0% (0/4 tests), because the zero bias leaves invalid states with no stable memory/attractor in the two-neuron network. This is a fundamental capacity limitation of minimal-size Hopfield networks, not an implementation problem.